

Session Report: July 2025

Summary

This report covers the work done across the July 2025 development session on the UAVX project. The work spans three main areas: the Ground Control Station (GCS), the Flight Controller (FC) firmware, and the simulation/test infrastructure. The overarching goal is to make the system safer, more configurable, and easier for pilots to set up and tune.

1. Parameter System Unification

What

Replaced the legacy uint8 parameter system with a unified float32 representation across both FC and GCS.

Why

The original FC stored parameters as uint8 values in flash and converted them to floats at runtime using per-parameter scale factors. The GCS had to reverse this process: take a human-readable display value, divide by the scale factor, and send the uint8 to the FC. This created two parallel code paths (legacy and float) that had to be kept in sync, and the conversion arithmetic was scattered across multiple files.

Changes

- **FC** (`params.h`, `params.c`): All 128 parameters now stored as float32 in `Config.ParamData[i].f`. The `ParamMetaEntry` table indexes by tag number. The old uint8 path is removed.
 - **GCS** (`parameters.py`): `PARAM_DISPLAY_MULT[idx]` converts FC floats to display values. No more `PARAM_SCALES` or `LEGACY_TAGS` needed at runtime.
 - **GCS** (`airframes.py`): `.af` files store the raw FC float values, not display values. The parser/formatter handles this transparently.
 - **GCS** (`parameter_window.py`): All combo boxes use `itemData()` with raw FC enum values rather than positional indices.
-

2. Airframe File Management

What

Reformatted all `.af` files for consistency, added `original_defaults/` as locked baselines, and established `*_Tuned.af` as the active working copies.

Why

The `.af` files were in mixed formats with inconsistent enum representations and no metadata. There was no way to compare a tuned configuration against the baseline, and no structure for storing physical aircraft descriptors (mass, arm length, prop size).

Changes

- `reformat_af.py`: Batch reformatted all 20 .af files for consistent formatting.
 - `original_defaults/`: 12 baseline .af files preserved as read-only reference. Locked until user says otherwise.
 - `parse_af()` **returns 3-tuple**: (name, values, metadata) — metadata captures `PHYS_*` keys and # Character: <n> slider position.
 - `format_af()` **accepts metadata dict**: Writes `PHYS_*` as key=value lines and other metadata as # Key: Value comments.
 - **Backward compatibility**: Old .af files with `PHYS_MASS_KG` are automatically converted to `PHYS_AUW_G` ($\times 1000$), `PHYS_ARM_LEN_M` to `PHYS_ARM_MM` ($\times 1000$).
-

3. Three-Layer Parameter Editor UX

What

Redesigned the parameter editor into three layers: Setup (required per-aircraft FC params), Physics (airframe descriptors), and Advanced (full 128-param grid, collapsed by default).

Why

The full 128-parameter grid was overwhelming for initial setup. Most pilots only need to configure 8-10 essential parameters (airframe type, ESC type, radio, battery, etc.) when setting up a new aircraft. Physical descriptors (mass, arm length, prop size) are pilot knowledge that doesn't belong in the FC parameter space.

Changes

- `_create_physics_group()`: New purple-bordered “Physics” QGroupBox. `AF_TYPE` combo on top with label after, dynamic physical descriptor grid below based on airframe category.
- **Setup box**: Contains essential FC params plus Character slider (conservative↔aggressive). `RX_TYPE` combo on top, `AF_TYPE` in Physics box, battery/ESC below, Character slider full-width at bottom.
- `_build_phys_row()`: 3-column grid layout that adapts per airframe category:
 - MR (5 fields): AUW, Arm, Prop, Motor W, Motors
 - FW (5 fields): AUW, Wing, Chord, Prop, Motor W
 - LAND (3 fields): AUW, Track/WB, Wheel dia
- `_compute_physics_from_descriptors()`: Computes derived quantities from physical descriptors: inertias, TWR, hover throttle, cruise speed, wing area, max thrust. Uses actuator disk theory with $\eta=0.50$ efficiency factor (ESC $\times$ motor $\times$ prop losses).
- **Advanced checkbox**: Toggles the full 128-param grid. RC Config and Motors/Servos groups have fixed height (160px) so the window doesn't stretch vertically.
- **Setup↔Advanced bidirectional sync**: Changes in Setup propagate to Advanced grid and vice versa via `_setup_sync_to_advanced()` and `sync_setup_from_advanced()`.
- **Save-before-losing-changes**: `_prompt_save_if_dirty()` prompts Save/Discard/Cancel when switching airframes, loading files, or closing window. Auto-saves to `~/UAVX/<airframe_name>_Tuned.af`.

Hover Throttle Calculation

The GCS computes expected hover throttle from physical descriptors:

$T_{\text{hover}} = \text{mass} \times g$ (thrust needed to hover)
 $v_{\text{induced}} = \sqrt{T_{\text{hover}} / (2 \times \text{wing area})}$ (induced velocity at prop disk)

```

P_hover = T_hover * v_induced          (mechanical power)
P_max = P_hover /                      (electrical power, =0.50)
hover_thr = P_hover / P_max            (throttle fraction)

```

Pilots should aim for 45-55% hover throttle. Outside 25-75% triggers a warning.

4. FW Aerodynamic Simulation Model

What

Implemented a full aerodynamic simulation for fixed-wing airframes in `test_pid_sim.py`, replacing the simplified MR-only model.

Why

The MR simulation used motor-thrust physics ($\text{torque} = \text{thrust} \times \text{arm} / \text{inertia}$) which is meaningless for fixed-wing aircraft. FW aircraft use aerodynamic control surfaces: ailerons produce roll torque from air pressure on wings, elevators produce pitch torque from tail surfaces, rudders produce yaw torque from vertical stabilizer deflection. The physics are fundamentally different.

Changes

- `run_physics_fw()`: Aerodynamic torque model: $\text{torque} = q_{\text{bar}} \times S \times C_{\text{ctrl}} \times \text{_surface}$ where $q_{\text{bar}} = 0.5 \times \rho \times V^2$ is dynamic pressure, S is wing area, and C_{ctrl} is the control derivative.
- **Quadratic damping**: $\text{damping} = C_d \times \text{rate}^2$ (aerodynamic drag scales with V^2) with critical damping sign correction: $\text{abs}(C_d)$ ensures damping opposes motion.
- **Linear damping at low rates**: $\text{damping} += C_{d_lin} \times \text{rate}$ (viscous effects at low Reynolds number).
- **Static stability moments**: Pitch stability from CG ahead of neutral point ($C_{m_} = -SM \times C_L$), yaw stability from vertical tail weathercock effect.
- **Cross-coupling**: Adverse yaw (roll→yaw), dihedral effect (yaw→roll via bank angle coupling).
- `simulate_freeflight()`: 15-second free-flight turbulence test with zero PID control — validates passive airframe stability (dihedral, static margin, weathercock).
- `simulate_axis_coupled()`: Full 3-axis coupled simulation with gusts applied simultaneously — tests real-world disturbance response.
- **Per-airframe physical descriptors** in `FW_AIRFRAMES` dict: mass, wingspan, wing area, control surface areas and arm lengths, control derivatives, damping coefficients, static stability derivatives.

Key Insight

FW PID step responses (10° pitch, 45° yaw) show large “failures” — this is correct behavior. A trimmed FW in level flight doesn’t need to hold a specific angle; it holds airspeed and altitude. The FC only intervenes for extreme attitudes (vertical climb/dive, inversion, stall). The simulation correctly shows that FW gains are conservative and disturbance rejection is the primary concern, not step tracking.

5. AH/Nav Step Response Simulation

What

Added altitude-hold and navigation step-response simulations for both MR and FW, using separate physics models appropriate to each airframe type.

Why

The existing attitude simulations test roll/pitch/yaw tracking, but altitude hold and navigation are the primary autopilot modes pilots care about. MR and FW have fundamentally different physics for these modes: - MR altitude: fast mass-thrust model (~0.5s response), throttle directly controls vertical force - FW altitude: slow pitch-to-climb model (~3s response), pitch angle trades airspeed for altitude via energy management - MR navigation: bank-to-turn with fast tilt (~0.3s), lateral accel = $g \times \tan(\text{bank})$ - FW navigation: coordinated turn with slower bank dynamics (~1.5s), heading rate = $g \times \tan(\text{bank})/V$

Bugs Fixed

1. `vel` **not initialized in** `simulate_nav_fw`: Variable used before assignment on first loop iteration — Python UnboundLocalError crash.
 2. `thr_lim = ALT_THROTTLE_COMP_LIMIT / 100.0`: The FC float is already a fraction (0.2 = 20%). Dividing by 100 made `thr_lim = 0.002`, essentially disabling the PI controller. MR AH sim produced 20m for a 5m step (pure open-loop drift from hover thrust slightly exceeding gravity).
 3. `critique()` **displaying meters as degrees**: AH/Nav output is in meters, but `critique()` multiplied all values by `RAD_TO_DEG` and appended °. Created `critique_linear()` for linear (meters) step response.
-

6. Character Slider (Conservative↔Aggressive)

What

Added a Character slider to the Setup group that interpolates between conservative (zero) and aggressive (100%) positions for 25 key parameters.

Why

Pilots new to UAVX need a simple way to adjust the overall “personality” of the tuning without understanding each individual parameter. The slider provides a single control that scales gains, limits, and damping in a coordinated way.

Changes

- `_PARAM_CURVES`: 25 parameter curves defining conservative/aggressive display values. Includes angle/rate PID gains, rate limits, altitude hold, navigation, and angle limits.
- `_get_scale_factors()`: Per-parameter scale factors based on physical aircraft properties. Rate gains scale with $1/\text{inertia}$, angle gains with mass, nav gains with $1/\text{mass}$. Reference aircraft: MR 800g/220mm/11”prop, FW 1000g/1800mm wingspan.
- `apply_slider()`: Interpolates between conservative and aggressive values at given slider position, applies display multiplier, and returns modified params dict.
- `compute_defaults()`: Applies slider position to base `_PARAM_CURVES` values, incorporating physics scaling. Called when slider is moved or Compute is pressed.

- `reset_to_computed()`: Reverts params AND restores slider position to last Compute snapshot.
- `_computed_slider_pos`: Stored on Compute so Reset knows where to revert to.

Validation

Slider extremes (0% and 100%) are tested in `test_pid_sim.py` for all `_Tuned.af` files. Conservative must not be unstable; aggressive must not exceed safety limits. Currently all MR airframes pass at aggressive; conservative shows slower response (no oscillation, all zero-crossings = 0). FW pitch/yaw step failures are expected (FW is trimmed cruise — doesn't track 30° pitch or 15° yaw commands).

Bug Fixes (v2)

Several issues were found and fixed during slider validation:

_PARAM_CURVES scaling bug: Values were stored in GCS display units but `_DISPLAY_MULT` dict was missing entries for most params (only had 6 of 27). `apply_slider()` treated display values as FC raw values, producing results off by factors up to 40,000× (e.g., `YAW_RATE_KD` conservative=100 → should have been 0.0025 raw). Fix: converted all curve values to FC raw float32 units (rad, rad/s, fraction), removed `_DISPLAY_MULT` dict entirely.

MR yaw damping coefficient: `run_physics_mr()` used hardcoded $2.0 \times \text{rate}^2$ for all axes, limiting yaw to 31°/s open-loop (`MAX_YAW_RATE` = 180°/s in tune). Fix: per-axis damping (Roll=0.015, Pitch=0.03, Yaw=0.05) providing realistic open-loop max rates (Roll ~360°/s, Pitch ~260°/s, Yaw ~200°/s).

Yaw angle limit clamp: `get_axis_params()` defaulted `MAX_YAW_ANGLE` to 0.524 rad (30°) since no yaw angle limit param exists on FC. This clamped the 45° yaw step setpoint, producing 30.1° final angle regardless of gains. Fix: default yaw max angle to 2π (360° continuous rotation).

GCS _PARAM_CURVES: Same unscaled values existed in `parameter_window.py`. `QDoubleSpinBox` widgets apply `PARAM_SCALES[idx]` automatically, but the curve display values were wrong (e.g., `ROLL_ANGLE_Q_INT_LIMIT` curve (0.3, 0.8) produced raw 0.000079-0.00021 instead of intended 0.005-0.03). Fix: recomputed all display values from raw FC targets using correct legacy/normal scale factors.

7. Safety-Critical Parameter Protection

What

Added confirmation dialogs in the GCS before changing safety-critical parameters.

Why

Certain parameters can cause catastrophic failure if set incorrectly: wrong airframe type causes mixer to send full throttle to wrong outputs; wrong ESC protocol causes motor runaway; wrong battery scale causes over-discharge. Pilots should be warned before changing these.

Changes

- `_PROTECTED_PARAMS`: 10 parameters requiring confirmation: `AFTType(43)`, `ESCType(35)`, `IdleThrottle(22)`, `Config1Bits(15)`, `VoltScale(85)`, `CurrScale(86)`, `ServoSense(51)`, `FW-BoardPitchAngle(81)`, `FWThrottleFF(64)`, `FWThrottleKp(65)`.

- `_PROTECTED_WARNINGS`: Specific danger text per parameter explaining what can go wrong.
 - `_rc_channel_changed()`: Detects when user assigns same RC channel to multiple functions and warns about clashes.
-

8. FC Firmware Changes

What

Several firmware improvements: yaw symmetry factor, nav yaw rate limiting, tilt throttle FF hardcoding, elevator/delta AF split, and drive symmetry enforcement.

Why

- **Yaw asymmetry**: Most MR frames have yaw authority limited by prop reactive drag, which scales with $D^{(5/3)}$. The old code used a fixed 0.6 factor; now configurable via `pYawSymmetryFactor` (default 0.8) with Config1 bit 7 to enable/disable enforcement.
 - **Nav yaw rate limit**: Without a limit, the nav controller can command unrealistic yaw rates that exceed the airframe's physical capability, causing oscillation. `Nav.MaxCompassRate` (30°/s default) limits this.
 - **Tilt throttle FF**: The old `pTiltThrFFfrac` was a tunable parameter that often got misconfigured. Now hardcoded to $1/\cos(\)$ — pure physics, not tunable. Slew limit provides sufficient LPF for Madgwick-filtered data.
 - **Elevator vs Delta AF split**: ElevonAF (bank-to-turn only, no rudder output) vs DeltaAF (rudder + aileron→rudder FF) — different mixing for different airframe configurations.
 - **FW FF params activated**: Tags 64 (FWThrottleFF), 65 (FWThrottleKp), 70 (FWYawFF), 86 (FWPitchFF) now have physics-computed defaults and mixer implementations. Bypassed in PassThru except `pFWBoardPitchAngleRad`.
-

9. Build System and Distribution

What

Set up GitHub repos, maintenance scripts, porting kits, and desktop launcher.

Why

The project needs a clean distribution path: primary directories are SVN working copies for version control, but GitHub is the public distribution channel. Build scripts need to handle PEP 668 restrictions on modern Linux distros, and users need a simple way to launch the GCS.

Changes

- **GitHub repos**: `gke/UAVXArmQ` (firmware binaries + wiki/docs) and `gke/UAVXGS` (3 platform kits + wiki/docs). No FC source code on GitHub.
- **Maintenance scripts**: 7 scripts in UAVXArmQ, 8 in UAVXGS. SVN commit, build, sync, push to GitHub. Scripts stay open on error or completion.
- `update_all.sh`: Full cycle — SVN commit → build/sync → update repo → push to GitHub. Single command for complete update.
- **Build scripts**: venv approach for PEP 668 compatibility. Error trap keeps terminal open on failure.

- **Desktop launcher:** ~/.local/share/applications/uavxgs.desktop with XGS icon. Uses venv python from run.sh.
 - run.sh: Clears __pycache__ across entire uavx-python/ tree.
-

10. Documentation

What

Created and organized wiki documentation with PDF generation.

Reports

- MC_TUNING_REPORT.md: Baseline simulation results for all 7 MR_Tuned airframes. Rise times 1.0-1.7s, overshoot 6-15%, disturbance rejection all PASS. Recommendations for gain adjustments.
- FW_SIMULATION_REPORT.md: Full tabulated results for 4 FW airframes. Free-flight stability, PID step response, disturbance rejection. Key insight: FW PID “failures” are correct physics — FW doesn’t track angles like MR.
- MULTICOPTER_SPECS_QUESTIONNAIRE.md: Template for gathering physical aircraft specs from pilots.
- Ecks_Airframe_Tuning.md: Detailed tuning guide for 4 Ecks configurations.

Organization

```
wiki/
  docs/      User-facing (GCS Guide, Conversion, What_Is_Different)
  reports/   Analysis (MC/FW sim reports, specs questionnaire)
  retired/   Legacy documentation
```

11. Physics Reference Values

Key physical constants and derived values used throughout the system:

Constant	Value	Source
Air density (sea level)	1.225 kg/m ³	ISA standard
Propulsive efficiency η	0.50	ESC(95%) × motor(80%) × prop(65%)
MR reference AUW	800g	Typical 220mm quad
MR reference arm	220mm	Motor-to-center distance
MR reference prop	11"	Typical freestyle prop
FW reference AUW	1000g	Typical 1800mm glider
FW reference wingspan	1800mm	Sky Surfer / Bixler class
Yaw authority scaling	$D^{(5/3)}$	Actuator disk theory for constant power

Session Fix Summary

Bug	Location	Root Cause	Fix
vel uninitialized	simulate_nav_fw()	Variable used before assignment on first loop iteration	Added vel = 0.0 initialization
thr_lim off by 100x Meters displayed as degrees	simulate_alt_hold_mr/fw() critique() for AH/Nav	FC float is already a fraction; /100.0 was wrong RAD_TO_DEG applied to linear quantities	Removed /100.0 Added critique_linear() for meters
AF_TYPE display wrong	parameter_window.py line 424, 502, 659	Raw enum value used as positional index	Changed to findData(int(value)) with positional fallback
RC channel writes 0	write_params() in parameter_window.py	QSpinBox (RC channel) not handled, fell through to QDoubleSpinBox branch	Added isinstance(widget, QSpinBox) branch
Missing _rc_channel_changed	parameter_window.py	Method referenced in lambda but never defined	Added method with dirty-flag + channel clash detection
SpeechLevel crash macOS build PEP 668 Linux build missing venv	main_window.py (4 copies) build_macos.sh build.sh	str(SpeechLevel.ALL) passed to QLabel.setText() System pip blocked by PEP 668 PEP 668 on Python 3.12+	Changed to SpeechLevel.ALL.value Added venv creation Added python3-venv + venv creation
_PARAM_CURVES scaling bug	test_pid_sim.py, parameter_window.py	Values in display units but _DISPLAY_MULT incomplete; apply_slider treated display as raw	Converted all curves to FC raw float units; removed _DISPLAY_MULT
MR yaw damping too high	test_pid_sim.py:run_physics_m2()	$0 \times \text{rate}^2$ hardcoded for all axes; yaw open-loop max = $31^\circ/\text{s}$	Per-axis damping: Roll=0.015, Pitch=0.03, Yaw=0.05
Yaw setpoint clamped at 30°	test_pid_sim.py:get_axis_param()	MAX_YAW_ANGLE defaulted to 0.524 rad (no FC param for yaw)	Default yaw max angle = 2π for 360° continuous
GCS slider display values wrong	parameter_window.py:_PARAM_CURVES	Angle int limit curves produced 0.8-2% of intended raw values	Recomputed from raw FC targets using correct PARAM_SCALES